

Real-Time Recurrent Learning Flight Surface Controller

A Complete Guide to the Project

An online-adapting neural flight controller
built and evaluated in two weeks

Project window: 27 July – 20 October 2026

Simulator: JSBSim (Cessna 172)

Core claim: online weight adaptation improves
fault tolerance in flight control

Contents

Part 1 — The Problem: Why Flight Control Is Hard

- What a flight controller actually does
- The three control surfaces
- Why classical control has limits
- Why neural networks are attractive here

Part 2 — The Gap: Frozen Brains in a Changing Sky

- How neural controllers are normally trained
- The frozen-at-deployment problem
- BPTT versus RTRL
- Why RTRL was impractical, and what changed

Part 3 — The Research Landscape

- Modern RTRL work
- Learning-based flight control
- Where this project sits

Part 4 — The Central Claim and How to Test It

- The one sentence the project defends
- Why the ablation is the real experiment
- The three competitors
- The four scenarios

Part 5 — System Architecture

- The layered picture
- Six design invariants
- How data flows through one timestep

Part 6 — Repository Tour

- Top-level orientation
- agent_docs — the intent layer
- configs — the experiment definitions
- src/env — the world
- src/controllers — the brains
- src/rtrl — the mathematical core
- src/training — how brains are made
- src/metrics and src/analysis — how we judge
- scripts — the entry points
- tests — the safety net
- data and report — the outputs

Part 7 — How the Project Grew

- Stage 1: Documents before code
- Stage 2: The mathematical core, verified first
- Stage 3: Discovering the simulator's real shape
- Stage 4: Building the world
- Stage 5: The three controllers
- Stage 6: The crisis — everything was broken
- Stage 7: The four bugs, in order
- Stage 8: The real fix
- Stage 9: Full sweep and results

Part 8 — What the Experiments Showed

- The headline result
- Reading the numbers honestly
- The five figures

Part 9 — Lessons, Limitations, and Next Steps

- What this project proves
- What it does not prove
- Engineering lessons worth keeping
- If you had four more weeks

Part 10 — Glossary

Part 1 — The Problem: Why Flight Control Is Hard

Before understanding what this project built, you need to understand what problem it is solving. This part assumes no aerospace background.

1.1 What a flight controller actually does

An aircraft in flight is a body moving through air, and air pushes back. Every gust, every change in speed, every shift in weight moves the aircraft away from where it is supposed to be. Something has to constantly correct for that. That something is the flight controller.

A flight controller is a loop that runs many times per second. Each cycle, it does three things:

1. **Read the state.** Where is the aircraft pointing? How fast is it rotating? How high is it? How fast is it going?
2. **Compare to the target.** Where *should* it be pointing?
3. **Command the surfaces.** Move the control surfaces to close the gap.

That is the whole job, stated simply. The difficulty is entirely in step 3: knowing *how much* to move each surface, given everything happening at once.

1.2 The three control surfaces

A conventional fixed-wing aircraft steers using three movable panels. Each one controls rotation about a different axis.

Surface	Axis	What it does physically
Aileron	Roll	Two panels on the trailing edges of the wings, moving in opposite directions. One wing gains lift, the other loses it, and the aircraft banks left or right.
Elevator	Pitch	A panel on the horizontal tail. Deflecting it changes the downward force on the tail, which tips the nose up or down.
Rudder	Yaw	A panel on the vertical tail. Pushes the tail sideways, swinging the nose left or right.

In this project each surface is commanded as a single number between -1 and $+1$. Zero means neutral, $+1$ means full deflection one way, -1 means full deflection the other. So the controller's entire output, every timestep, is three numbers.

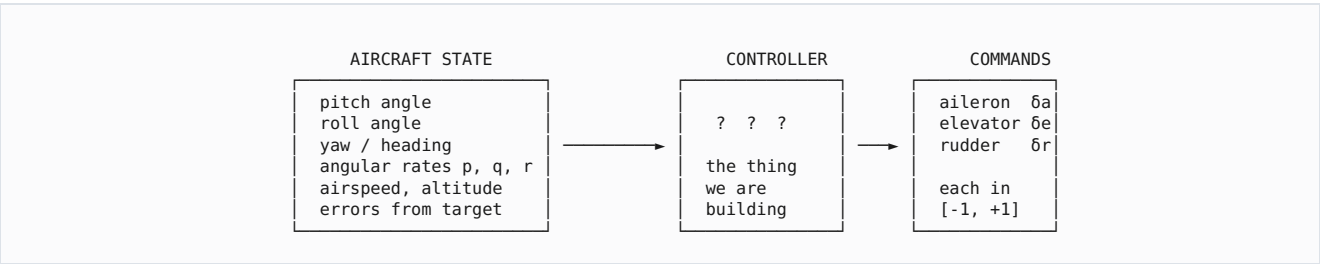


Figure 1.1 — The control problem, reduced to inputs and outputs.

1.3 Why classical control has limits

The traditional answer to this problem is the **PID controller** — Proportional, Integral, Derivative. It is a simple formula: look at the current error, add up the error over time, look at how fast the error is changing, and combine those three with fixed weights called *gains*.

PID is remarkable. It has flown aircraft for decades, it is easy to reason about, and it is provably stable within its operating envelope. This project uses it as a baseline, and — as you will see later — a properly tuned PID turned out to be the strongest performer in the whole study.

But PID has three real limitations:

- **It has no memory of shape.** PID responds to the error right now and the error's recent slope. It does not recognise patterns. It cannot notice "this looks like the oscillation that happened thirty seconds ago."
- **Its gains are fixed.** A gain tuned for cruise at 5,000 feet is wrong for a slow climb or a steep bank. Real aircraft handle this with gain scheduling — swapping gain tables based on flight condition — which is a large manual

engineering effort.

- **It assumes the aircraft doesn't change.** If an aileron loses half its effectiveness, PID keeps applying the same gain to a surface that now does half as much. It will eventually compensate through the integral term, but slowly and often with overshoot.

1.4 Why neural networks are attractive here

A neural network is a function that can be shaped by data. Feed it enough examples of "state in, correct command out," and it learns the mapping — including nonlinear relationships that a PID's fixed formula cannot express.

A *recurrent* neural network goes further. It carries an internal state — a hidden vector — from one timestep to the next. This means it can, in principle, remember. It can build up an internal sense of how the aircraft has been behaving over the last few seconds, not just what it is doing this instant.

THE KEY IDEA OF THIS PROJECT

Memory is only half of what a recurrent network could give a flight controller. The other half — the half almost nobody uses — is the ability to *keep learning while flying*. That is what Real-Time Recurrent Learning enables, and that is what this project is about.

Part 2 — The Gap: Frozen Brains in a Changing Sky

Almost every neural flight controller in the published literature shares the same limitation. Understanding that limitation is understanding why this project exists.

2.1 How neural controllers are normally trained

There are two standard recipes.

Recipe one: imitation. Take an existing controller — a PID, or a human pilot's recorded inputs — and train the network to copy it. Show the network thousands of examples: "when the state looked like this, the expert did that." The network learns to reproduce the expert's behaviour.

Recipe two: reinforcement learning. Let the network fly in simulation, give it a reward signal (high reward for good tracking, low for bad), and use an algorithm like PPO or SAC to nudge its weights toward higher reward. Repeat for millions of steps.

Both recipes work. Both produce controllers that fly. And both share a fatal structural property.

2.2 The frozen-at-deployment problem

In both recipes, training happens first, and flying happens after. Once the network is deployed — put in an actual aircraft, or evaluated in a test episode — **its weights stop changing**. The network is frozen.

Think about what that means. During training, the network saw simulated wind, simulated failures, simulated turbulence. It learned a policy that handles the average of what it saw. But in the air:

- A gust arrives with a profile it never trained on. The frozen network responds with its best average guess.
- An aileron actuator degrades to 50% authority. The frozen network keeps commanding as though it were at 100%.
- The aircraft's mass distribution shifts as fuel burns. The frozen network does not notice.

In every case, the network has all the information it needs — it can see the tracking error growing — but it has no mechanism to change its own behaviour in response. It can only react through the input-output mapping it was frozen with.

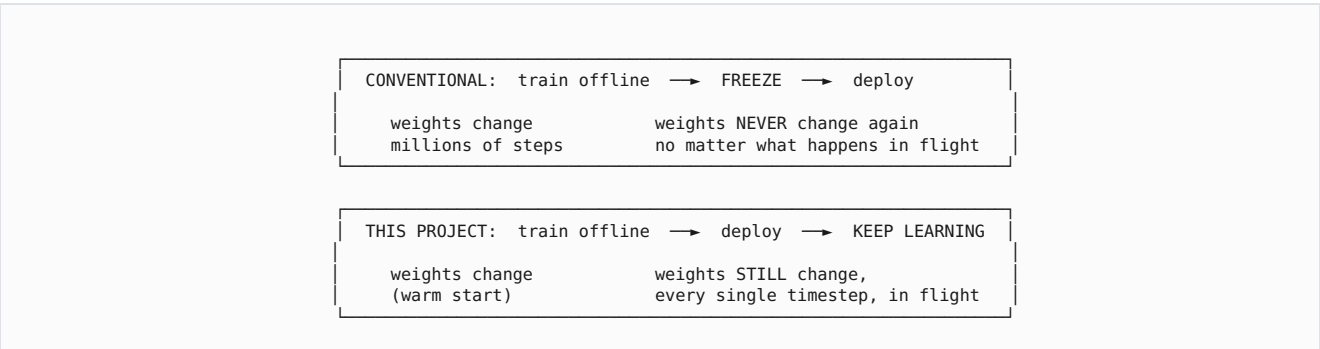


Figure 2.1 — The structural difference this project explores.

2.3 BPTT versus RTRL

Why do conventional methods freeze? Because of the algorithm they use to learn: **Backpropagation Through Time**, or BPTT.

BPTT works by unrolling the network's history. To compute how a weight should change, it needs to know how that weight affected the hidden state at step 1, which affected step 2, which affected step 3, all the way to the present. So BPTT stores the entire sequence, then walks *backward* through it computing gradients.

This has two consequences. First, you must store the history — memory grows with sequence length, so in practice everyone truncates to a window of maybe 20 or 50 steps and accepts the approximation. Second, and more important: **you cannot learn until the sequence is over**. BPTT is fundamentally retrospective. It looks backward from the end.

Real-Time Recurrent Learning — RTRL — inverts this. Instead of walking backward at the end, it carries the gradient information *forward*, alongside the network's normal computation. At every timestep it maintains an extra quantity, the *sensitivity matrix*, which answers: "how would the current hidden state change if I nudged each weight?"

Property	BPTT	RTRL
Direction of computation	Backward, at the end	Forward, as it goes
Needs stored history	Yes	No
Context length	Truncated in practice	Unbounded — exact
Can learn mid-episode	No	Yes
Cost per step	Cheap	Historically prohibitive

Because RTRL carries its gradient information forward, it can update the weights at any timestep it likes — including in the middle of a flight, in response to something that just happened. That single property is the entire foundation of this project.

2.4 Why RTRL was impractical, and what changed

RTRL was published in 1989. It has been largely unused for thirty-five years because of one number: its cost.

The sensitivity matrix has one entry for every combination of (hidden unit, weight). For a fully-connected recurrent network with n hidden units, there are roughly n^2 weights, so the matrix has n^3 entries, and updating it each step costs on the order of n^4 operations. For a network of any useful size, that is hopeless.

What changed is a realisation that arrived properly in 2023–2024: **if the recurrence is element-wise, the cost collapses.**

In a standard recurrent network, every hidden unit talks to every other hidden unit. That dense coupling is what creates the n^4 cost. But if you restrict the architecture so each hidden unit evolves independently — each unit's next value depends only on its own previous value plus the current input — then most of the sensitivity matrix is structurally zero. You only need to track the diagonal. The cost drops to something entirely manageable.

The trade-off is expressiveness: you lose the dense hidden-to-hidden coupling. Recent work (Irie et al. 2024, Elelimy et al. 2024) established empirically that this trade-off is often worth it — these restricted architectures perform competitively, sometimes better, than dense ones, while making exact RTRL practical.

WHY THIS MATTERS FOR THIS PROJECT

The architecture used here is a **Recurrent Trace Unit (RTU)** — an element-wise recurrent cell where each hidden unit has its own decay factor and evolves independently. This is what makes RTRL affordable at 5 Hz on a laptop. The architecture choice is not incidental; it is what makes the whole approach possible.

Part 3 — The Research Landscape

This section places the project among existing work. It is deliberately brief — enough to see the gap the project fills.

3.1 Modern RTRL work

Work	Contribution
Irie et al., ICLR 2024 <i>Promise and Limits of RTRL</i>	Showed that element-wise recurrent architectures make exact RTRL tractable with no approximation. Their actor-critic system beat established baselines using far fewer environment steps.
Elelimy et al., NeurIPS 2024 <i>Recurrent Trace Units</i>	Introduced RTUs — a small modification to linear recurrent units that makes RTRL cheap and outperforms GRUs and Transformers on partially observable tasks. This is the architecture family used in this project.
Lemmel & Grosu, AAAI 2025 <i>Real-Time Recurrent RL</i>	Combined RTRL with temporal-difference learning and eligibility traces for partially observable control.
Lam, Sirignano & Spiliopoulos, 2025	Convergence proofs for RTRL on a class of recurrent networks.

3.2 Learning-based flight control

Approach	Nature and limitation
PPO / SAC on JSBSim	The dominant approach. Trains a policy through simulated rollouts, then deploys frozen. Effective, well-studied, but cannot adapt in flight.
LSTM fault-tolerant control	Uses recurrence for memory, trained with BPTT. Handles faults it saw in training; struggles with novel ones. Also frozen at deployment.
Incremental Dual Heuristic Programming (TU Delft)	The closest existing relative. Learns an online local model of the aircraft and adapts in flight. But it has no recurrent memory — it fits a local linear model each step, with no notion of longer temporal structure.
Neural-Fly (Caltech)	Meta-learning plus online Kalman adaptation for wind. State of the art for wind-robust quadrotor control, but rotorcraft rather than fixed-wing, and the adaptation is over a small set of learned basis coefficients rather than the full network.

3.3 Where this project sits

Reading those two tables together, a specific gap appears:

THE GAP

BPTT-trained controllers have **memory but no adaptation**. IDHP-style controllers have **adaptation but no memory**. Nobody has published a fixed-wing flight surface controller that has **both** — recurrent memory *and* genuine online weight updates — using modern efficient RTRL.

That is the space this project occupies. It is not a claim to have beaten the state of the art on tracking accuracy. It is a claim to have built and honestly evaluated something in a corner of the design space that has been left empty because the enabling algorithm only became practical two years ago.

Part 4 — The Central Claim and How to Test It

4.1 The one sentence the project defends

THE CLAIM

A recurrent flight controller that continues updating its weights during flight, using Real-Time Recurrent Learning, recovers from mid-flight actuator degradation better than the same controller with those updates switched off.

Read that carefully, because the precision matters. It does **not** claim RTRL beats PID. It does **not** claim RTRL beats an LSTM. It claims that *online adaptation* — the specific thing RTRL uniquely enables — produces measurably better fault recovery than no online adaptation, holding everything else constant.

4.2 Why the ablation is the real experiment

This design choice is the intellectual heart of the project, so it is worth dwelling on.

Suppose you build an RTRL controller and it outperforms an LSTM. What have you shown? Almost nothing, unfortunately. The two differ in architecture, training procedure, hyperparameters, and amount of tuning effort. Any of those could explain the difference. The comparison is confounded six ways.

Now suppose instead you take *one* RTRL controller, with *one* set of weights, and run it twice — once with online updates enabled, once with them disabled. Same architecture. Same weights at the start of the episode. Same environment. Same seed. The *only* difference is whether the weights are allowed to change during flight.

If the online-enabled version does better, there is only one explanation available. That is a clean experiment.

CONFOUNDED COMPARISON			CLEAN ABLATION		
RTRL-RTU	vs	BPTT-LSTM	RTRL-RTU online ON	vs	RTRL-RTU online OFF
diff arch		diff arch	same arch		same arch
diff train		diff train	same weights		same weights
diff params		diff params	same env		same env
diff tuning		diff tuning	same seed		same seed
→ difference explains nothing			→ difference explains ONE thing		

Figure 4.1 — Why the ablation, not the cross-controller comparison, is the real experiment.

4.3 The three competitors

Three controllers are built and run through every scenario.

Controller	What it is	Role in the study
PID	Classical control, fixed gains	The floor. If a neural approach cannot beat a well-tuned PID, that must be stated plainly. It also serves as the teacher for imitation training.
BPTT-LSTM	Recurrent network, trained by imitation, frozen at deployment	The conventional neural baseline — representing how essentially all published neural flight controllers work.
RTRL-RTU	Element-wise recurrent network, updating weights every timestep	The experimental method. Warm-started from the LSTM, then allowed to keep learning in flight.

4.4 The four scenarios

Scenario	What it tests
1. Nominal	Calm air, healthy aircraft. Establishes that each controller can fly at all. A controller that fails here tells you nothing useful in the harder scenarios.
2. Fault	At step 150 — mid-flight — one control surface drops to 50% effectiveness. The aircraft is now physically different from the one every controller trained on. <i>This is the headline scenario.</i>
3. Wind	Dryden turbulence at a severity never seen during training. Tests generalisation to unseen disturbances.
4. Combined	Fault and turbulence simultaneously. The stress test.
5. Ablation	The fault scenario again, but with RTRL's online updates disabled. The controlled comparison described above.

Part 5 — System Architecture

5.1 The layered picture

The system is built in layers, each with one responsibility, each knowing as little as possible about the others.

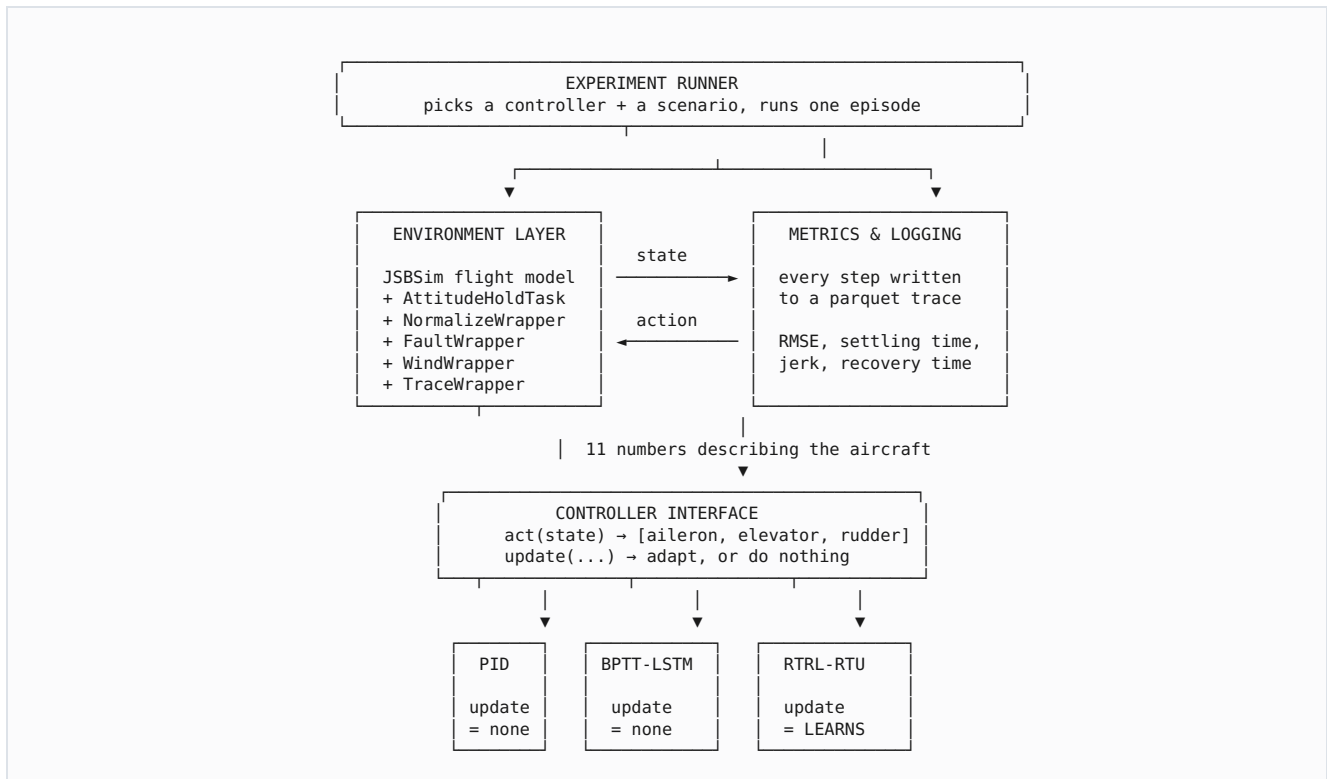


Figure 5.1 — The complete system, top to bottom.

5.2 Six design invariants

Six rules were fixed at the start and never violated. Each one prevented a specific class of mess.

Invariant 1 — All controllers share one interface

Every controller answers the same four questions: give me an action, update yourself, reset for a new episode, save/load your state. The experiment runner does not know which controller it is holding.

What this prevents: a runner littered with special cases. Adding a fourth controller would require zero changes to the runner.

Invariant 2 — Faults and wind live in wrappers

A fault is not something the controller knows about. It is a modification of the world, implemented as a layer that sits between the controller's command and the simulator, quietly scaling it down.

What this prevents: controllers that cheat by knowing when a fault occurred. It also makes the scenarios compose — wind and fault stack cleanly in either order.

Invariant 3 — Online learning is a method on the controller

The runner calls `update()` after every step, always, for every controller. For PID and the frozen LSTM, that call does nothing. For RTRL, it performs a learning step.

What this prevents: the runner having to know that RTRL is special. The ablation becomes a single configuration flag, not a separate code path.

Invariant 4 — All data flows through the trace logger to disk

Every step of every episode is written to a parquet file. Analysis code reads those files. It never touches live simulation state.

What this prevents: results you cannot reproduce. Every number in the final report can be traced to a file on disk.

Invariant 5 — One configuration file per experiment

Nothing is hardcoded. Every run records the exact configuration it used, alongside its seed and git commit.

What this prevents: the classic disaster of a good result you can never reproduce because you changed a number and forgot.

Invariant 6 — Library code never imports from scripts

Dependencies flow one way: scripts depend on the library, never the reverse.

What this prevents: circular imports and the slow slide into a codebase where nothing can be tested in isolation.

5.3 How data flows through one timestep

Following a single timestep end to end makes the architecture concrete.

Step	What happens
1	The simulator reports the aircraft's raw physical state: altitude, pitch, roll, three velocity components, three rotation rates.
2	The task layer computes two error values — how far pitch and roll are from their targets — and appends them. The state is now 11 numbers.
3	The normalize wrapper rescales all 11 into a -1 to $+1$ range, so the controller never sees a raw number like 5,000 next to a number like 0.03.
4	The controller receives those 11 numbers and produces 3 commands.
5	If a fault is active, the fault wrapper silently scales the affected command down. The controller is not told.
6	The simulator applies the commands and advances the physics.
7	A reward is computed from the new tracking error, rotation rates, and control effort.
8	The trace logger records everything — state, commands, reward, fault status, wind level — as one row.
9	The runner calls <code>update()</code> . For RTRL, this is where learning happens: the sensitivity matrix is used to nudge the weights. For the others, nothing.
10	Repeat, 5 times per second, for 299 steps — about 60 seconds of simulated flight.

Part 6 — Repository Tour

This part walks through every folder in the project and explains what it is for and why it exists. No code, just purpose.

6.1 Top-level orientation

```
rtrl-flight-controller/  
|  
├─ CLAUDE.md           ← the rules of engagement  
├─ pyproject.toml      ← dependencies, tool settings  
├─ README.md           ← setup and quickstart  
|  
├─ agent_docs/         ← WHY things are the way they are  
├─ configs/            ← WHAT to run  
├─ src/rtrl_flight/    ← the actual library  
├─ scripts/            ← HOW to run it  
├─ tests/              ← proof it works  
├─ notebooks/         ← exploration  
├─ data/               ← generated outputs (not in git)  
└─ report/             ← figures and tables for writing up
```

The separation is deliberate. **Documents describe intent. Configs describe experiments. Source describes mechanism. Scripts describe invocation. Tests describe correctness.** Each answers a different question, and mixing them is how projects become unmaintainable.

CLAUDE.md — the rules of engagement

This file is unusual and worth explaining, because it shaped how the whole project was built.

It is a document written for whoever — or whatever — works on the codebase next. It states the project's goal in two sentences, lists the design invariants, and then, crucially, contains a section titled **"What NOT to do."**

That section says things like: do not add other aircraft, do not add trajectory tracking, do not add PPO baselines, do not "improve" the RTRL update with tricks from papers we have not discussed, do not skip the imitation warm-start.

Why does a project need a file of prohibitions? Because in a two-week window, the greatest danger is not doing the work badly — it is doing *additional* work. Every appealing extension ("we should also try the F-16," "we should add a PPO baseline for comparison") is a day you do not have. The prohibitions were written on day one, before any of those temptations arose, when it was easy to be disciplined.

LESSON WORTH STEALING

Writing down what you will *not* build, at the start, is more valuable than writing down what you will. The build list changes as you learn. The scope boundary should not.

6.2 agent_docs/ — the intent layer

Seven markdown documents, written *before* the corresponding code. Each one is the source of truth for a slice of the system's design.

Document	What it holds
project_overview.md	The goal, the two-week scope, what counts as success, and the order in which things get built.
architecture.md	The system diagram, the six invariants, and the contract each component must satisfy.
environment.md	The simulator setup, exactly what the 11 state numbers mean, the fault and wind models, the reward function.
controllers.md	Each controller's design, structure, and hyperparameters.
rttl.md	The mathematics — the sensitivity recursion, the update rule, why the architecture must be element-wise.
experiments.md	The five scenarios, precisely specified, written before any experiment code existed.
evaluation.md	Every metric, defined unambiguously. What counts as "settled." What counts as "recovered."
research_context.md	The related work and what this project claims relative to it.

WHY DOCUMENTS CAME FIRST

There is a hard test buried here. If you cannot write `experiments.md` — a precise statement of what you will run and what you will measure — *before* writing the experiment code, then you do not yet know what your experiment is. Writing the document forces the thinking. It also means the eventual report's Experiments section is already 80% written.

These documents were also kept honest. During the final health check, three of them were found to have drifted from the built system — an error sign convention documented backwards, a description of an update rule that had since been replaced, a statistical method that differed from what was implemented. All three were corrected. A design document that lies is worse than no document.

6.3 configs/ — the experiment definitions

```

configs/
├─ config.yaml           ← top-level; deliberately empty of defaults
├─ controller/
│   ├─ pid.yaml          ← gains for each surface
│   ├─ bptt_lstm.yaml    ← hidden size, layers, learning rate
│   └─ rttl_rtu.yaml     ← hidden size, online learning rate, ablation switch
├─ env/
│   ├─ cessna172_nominal.yaml ← scenario 1
│   ├─ cessna172_fault.yaml  ← scenario 2: surface, severity, onset step
│   ├─ cessna172_wind.yaml   ← scenario 3: turbulence level
│   └─ cessna172_combined.yaml ← scenario 4
└─ experiment/
    ├─ nominal_sweep.yaml
    ├─ fault_sweep.yaml
    ├─ wind_sweep.yaml
    └─ ablation_online_off.yaml ← scenario 5

```

Every experimental variable lives here rather than in code. Which aircraft, which surface fails, how badly, at what step, how strong the turbulence, which random seeds.

One design decision is worth noting. The top-level config deliberately declares *no* default controller, environment, or experiment. Every run must name all three explicitly on the command line. This means no run can silently inherit a setting nobody chose — a small discipline that eliminates a whole category of "why did this run behave differently?" confusion.

6.4 src/env/ — the world

This is the layer that stands between the controller and the physics engine. It has two jobs: define the task, and modify the world for each scenario.

The task definition

The underlying simulator, JSBSim, is a high-fidelity flight dynamics model — it computes real aerodynamics for a Cessna 172. Its Python wrapper ships with two ready-made tasks: hold a heading, and turn to a heading.

Neither is what this project needs. This project needs *attitude hold*: given a target pitch angle and a target roll angle, keep the aircraft at those angles. That task did not exist, so it was written.

The task defines three things: what the controller can see (the 11 numbers), what it can do (the 3 commands), and what counts as good (the reward function). Two of the 11 observation slots are the task's own contribution — the current error between target and actual attitude, which is the most directly useful signal the controller receives.

The reward function

The reward is a weighted penalty with three parts:

- **Tracking error** — how far pitch and roll are from target. This dominates.
- **Rotation rates** — penalises spinning, which discourages oscillation.
- **Control effort** — a small penalty on large commands, discouraging violent inputs.

The four wrappers

A wrapper is a layer that sits around the environment, intercepting information flowing in or out. Four are used, and they stack.

Wrapper	Job
Normalize	Rescales all state values into -1 to $+1$. Without this, altitude ($\sim 5,000$) sits next to pitch error (~ 0.1) and the network is dominated by the largest number.
Fault	Holds an effectiveness value per surface, starting at 100%. When triggered mid-episode, it silently scales that surface's command down. Resets to full on a new episode.
Wind	Configures JSBSim's built-in turbulence model at one of four severity levels. Purely atmospheric — never touches the commands.
Trace	Records every step to disk. The outermost layer, so it can see everything the others did.

WHY WRAPPERS RATHER THAN OPTIONS

Faults and wind could have been flags on the environment. Making them separate layers means they compose automatically — the combined scenario required no new code, just stacking two wrappers. It also means the fault logic is written once and tested once, rather than tangled into the environment's main loop.

6.5 src/controllers/ — the brains

Three controllers, one shared contract.

The contract

Every controller must answer four questions:

- **act** — given the state, what are your three commands?
- **update** — here is what just happened; adapt if you can.
- **reset** — a new episode is starting; clear your episode state but keep what you learned.
- **save / load** — write yourself to disk, restore yourself from disk.

The distinction inside `reset` matters. A controller has two kinds of internal state: *episode state* (the hidden vector, accumulated errors — things that are about this flight) and *learned state* (the weights — things learned across flights). Reset clears the first and preserves the second.

PID

Three independent loops, one per surface, each combining current error, accumulated error, and error rate. Its `update` does nothing — PID does not learn.

An important note: PID is not just a baseline here. It is also the *teacher*. The LSTM learns by imitating it, and the RTRL controller is warm-started from the LSTM. So PID's quality propagates through the entire chain — a fact that turned out to matter enormously, as Part 7 describes.

BPTT-LSTM

A two-layer recurrent network with 64 hidden units per layer. It is trained by imitation — shown thousands of examples of PID's behaviour — and then frozen. Its `update` deliberately does nothing.

That deliberate nothing is the point. This controller *represents* the conventional approach. Making it adapt would destroy its value as a baseline.

RTRL-RTU

The experimental controller. An element-wise recurrent network where each hidden unit has its own decay factor and evolves independently — the structural property that makes exact RTRL affordable.

Its lifecycle has three phases:

- 1. **Warm start.** It inherits weights from the trained LSTM. It does not begin from noise.
- 2. **Online training.** It flies 50 episodes, updating its weights at every step.
- 3. **Deployment.** It flies the evaluation scenarios, still updating every step — unless the ablation flag disables that.

WHY THE WARM START IS MANDATORY

Starting online RTRL from random weights on real flight dynamics diverges — reliably. The aircraft leaves controlled flight before the learning signal can accumulate. The warm start is not an optimisation; it is a precondition. This is written into `CLAUDE.md` as a prohibition specifically so nobody later "simplifies" it away.

6.6 src/rtrl/ — the mathematical core

The smallest folder in the project and by far the most important. Everything else can be wrong and fixed; if this is wrong, nothing above it means anything.

Component	Purpose
RTU cell	The recurrent unit itself. Each hidden unit keeps a fraction of its previous value and adds a contribution from the current input. The independence between units is the whole basis of tractability.
Sensitivity recursion	The heart of RTRL. Maintains, and updates each step, the answer to: "how would the current hidden state change if each weight were nudged?"
Online update	Combines the sensitivity information with the current error to produce a weight change.
Warm start	Transfers the LSTM's learned mapping into the RTU's differently-shaped parameters.

The most important test in the project

There is one test that everything else rests on. It verifies that the forward, step-by-step sensitivity computation produces the same gradients as two independent references: a full backward unroll, and a brute-force finite-difference approximation. Agreement to about five decimal places.

Why does this deserve such prominence? Because RTRL is easy to get *plausibly* wrong. A subtly incorrect sensitivity recursion still runs. It still produces numbers. Training still appears to proceed. You could build the entire project on top of it and never notice — you would simply be doing something that resembles RTRL rather than RTRL.

THE CORRECTNESS FLOOR

This test was written and passing on day five, before RTRL touched the flight simulator. It was re-run before and after every subsequent change to the mathematical core, and never allowed to fail. `CLAUDE.md` names it explicitly as non-negotiable.

6.7 src/training/ — how brains are made

Component	Purpose
Imitation	Runs PID for many episodes, records every state and the command PID chose, then trains the LSTM to reproduce those choices.
Online training	Loads the warm-started RTU and flies it for 50 episodes with learning enabled every step. Reports episode return and sensitivity-matrix magnitude — the latter being the early-warning signal for instability.

Note the dependency chain: **PID quality → imitation quality → LSTM quality → warm-start quality → RTRL's starting point.** A weakness anywhere upstream propagates all the way down. Part 7 tells the story of what happened when the very first link in that chain turned out to be broken.

6.8 src/metrics/ and src/analysis/ — how we judge

Metrics turn a recorded episode into numbers. Analysis turns many episodes into tables and figures.

Metric	What it captures
Attitude RMSE	Average tracking error across the episode. The primary quality measure — lower is better.
Settling time	How long until the error drops below a threshold and stays there.
Episode return	Total accumulated reward. A combined measure of tracking, smoothness, and effort.
Control jerk	How violently the commands change step to step. Low is smooth. But note: <i>zero is a red flag</i> , not perfection — it means the controller is outputting a constant, which means it has stopped responding to input at all.
Time to recover	Steps from fault onset until tracking error returns below threshold and stays there. The single most important number in the fault scenario.

A METRIC THAT EARNED ITS KEEP

Control jerk was included as a smoothness measure. It ended up being the diagnostic that caught the project's most serious bug. A jerk of exactly 0.0000 revealed that the RTRL controller had collapsed into emitting one constant command forever — a failure that the RMSE number alone made look merely "poor" rather than "completely broken."

6.9 scripts/ — the entry points

Script	Purpose
probe_jsbgym.py	A throwaway diagnostic, written before any real code, to discover what the simulator actually provides rather than assuming.
tune_pid.py	A diagnostic for measuring PID's actual tracking error, used during the emergency retuning.
train_bpitt.py	Runs imitation training for the LSTM.
train_rtrl_warmstart.py	Warm-starts and online-trains the RTU.
run_experiment.py	Runs one controller through one scenario, prints a metric summary.
run_sweep.py	Runs the full matrix — 39 episodes across all scenarios, controllers, and seeds — and saves the results table.
make_plots.py	Generates the five report figures from saved results.
live_demo.py	The interactive dashboard.

Scripts contain no logic. They parse arguments, call into the library, and print. All the substance lives in `src/`, where it can be tested.

The live dashboard

A four-panel window that updates in real time as an episode runs: attitude tracking (target versus actual), control surface commands, RTRL diagnostics (learning loss and sensitivity magnitude), and a text panel showing fault and wind status.

Keyboard keys trigger faults and cycle wind severity mid-flight. This makes for a compelling demonstration — "watch what happens when I jam the aileron *right now*" — but its real value was as a debugging tool. Watching the sensitivity magnitude climb in real time makes a divergence obvious in seconds, where the same problem hidden in a log file might take an hour to find.

WHAT WAS DELIBERATELY NOT BUILT

A 3D animated view of the aircraft was considered and rejected. It would have taken days, slowed simulation to real time, and shown nothing the graphs do not show more precisely. Examiners of a machine-learning project want learning curves, not scenery. This was one of the better scope decisions in the project.

6.10 tests/ — the safety net

Forty-four tests across five files.

Test file	What it protects
<code>test_rtrl_sensitivity.py</code>	The correctness floor. Verifies the RTRL mathematics against two independent references.
<code>test_controller_interface.py</code>	Every controller satisfies the shared contract.
<code>test_env_wrappers.py</code>	Faults apply correctly, wind toggles, traces are written, wrappers compose and reset cleanly.
<code>test_bptt_lstm.py</code>	Output bounds, hidden state behaviour, save/load fidelity, and that update genuinely does nothing.
<code>test_rtrl_rtu.py</code>	The same, plus that the ablation flag truly disables learning.
<code>test_metrics.py</code>	Metrics compute what they claim on hand-constructed data.

The environment tests run against the real simulator rather than a mock. Mocking would defeat the purpose — the entire question is whether the wrappers behave correctly against JSBSim's actual interface, which a fake cannot exercise.

6.11 data/ and report/ — the outputs

`data/` holds generated artefacts: per-episode traces, trained model checkpoints, and the aggregated results table. None of it is in version control — it is all reproducible from code plus configuration.

`report/` holds the five figures and the results table, both generated programmatically. Nothing there is typed by hand, which means nothing there can silently disagree with the underlying data.

Part 7 — How the Project Grew

This is the most instructive part of the document. The project did not proceed smoothly. Understanding where it broke, and why, teaches more than the final architecture does.

7.1 Stage 1 — Documents before code

Before a single line of implementation, eight markdown documents and one `CLAUDE.md` were written. Scope, architecture, mathematics, scenarios, metrics, and an explicit list of things that would not be built.

This felt slow. It was not. The scenarios written on day one were the scenarios run on day fourteen, unchanged. The scope boundary held. And when the project later hit its crisis, the documents provided a fixed reference for what the system was *supposed* to do, which made diagnosis dramatically faster.

7.2 Stage 2 — The mathematical core, verified first

The very first code written was the RTU cell and the sensitivity recursion — and immediately after, the test verifying them against a full backward unroll and finite differences.

This ordering was deliberate. RTRL is the project's one genuinely novel component and its one irreplaceable dependency. If it were wrong, everything built on top would be worthless. Verifying it in isolation, on a tiny toy problem, before any simulator was involved, meant that from that point forward the mathematics was never in question. Every subsequent bug could be attributed elsewhere with confidence.

ORDERING PRINCIPLE

Build and verify the component that everything else depends on, and that would be hardest to debug in situ, *first* — in the simplest possible setting.

7.3 Stage 3 — Discovering the simulator's real shape

Next came a throwaway script whose only purpose was to ask the simulator what it actually provides — what state variables exist, what their ranges are, what tasks are available.

This found a significant surprise. The design document had assumed a ready-made attitude-hold task existed. It did not. The library provides heading control and turn-to-heading, and nothing that tracks a target pitch and roll.

Two of the eleven observation slots were also wrong: the document expected pitch and roll *errors*, but the built-in task supplies altitude and heading errors instead — useful for a different problem entirely.

The document was corrected to match reality before any code was written against it. Had the probe been skipped, several days of work would have been built on a false assumption.

LESSON

Verify your dependency's actual interface before designing against it. The cost is thirty minutes. The cost of skipping it can be days.

7.4 Stage 4 — Building the world

With the real interface known, the environment layer went in cleanly: the custom attitude-hold task, the reward function, and the four wrappers. Twelve tests were written alongside. This stage produced no surprises, which is what good preparation looks like.

7.5 Stage 5 — The three controllers

PID, then the LSTM, then RTRL. Each satisfied the shared contract; each got its own tests. By this point the suite stood at thirty-five tests, all passing. The system appeared healthy.

7.6 Stage 6 — The crisis

Then the first real experiments ran, and the results were catastrophic.

Controller	Attitude RMSE	Control jerk	Interpretation
PID (nominal)	1.58 rad	1.15	Not controlling the aircraft
BPTT-LSTM (nominal)	1.90 rad	0.014	Copying a broken teacher
RTRL-RTU (nominal)	2.04 rad	0.0000	Emitting a constant. Dead.

For context: the targets range over roughly ± 0.3 radians in pitch and ± 0.4 in roll. An error of 1.5 to 2 radians means the aircraft is not being controlled in any meaningful sense.

Worse, the RTRL controller's jerk of exactly zero meant it was outputting the same three numbers on every timestep of every episode, regardless of input. It had collapsed into a constant function.

Every test still passed. Forty-four green. The tests verified that each component did what it claimed — and each component *did*. The system was correct and useless simultaneously. This is a distinction worth internalising.

7.7 Stage 7 — The four bugs, in order

Bug 1 — The PID's sign was inverted

Investigation began at the bottom of the dependency chain: the PID. Reducing the gains did not help. Increasing them made divergence faster. That pattern — where *less* control authority is better and *more* is worse — is the signature of a sign error, not a tuning problem.

An open-loop test confirmed it: in this simulator's convention, a positive elevator command pitches the nose *down*, while the controller was applying gain as though it pitched *up*. Every correction was actively making things worse.

A second, smaller error surfaced alongside it: the PID's assumed timestep was 0.02 seconds, but the simulation actually steps at 0.2 seconds. A factor of ten wrong, corrupting the integral and derivative terms.

After fixing the sign, adding a rudder coordination term, correcting the timestep, and retuning: **pitch error dropped from divergent to 0.115 radians, roll error to 0.050**. Episode return went from $-4,935$ to -12.5 .

THE TEACHING POINT

No amount of gain tuning can fix a sign error. When more control authority makes things worse, stop tuning and check the direction. Also note what enabled the fix: an *open-loop* test — command a fixed deflection, observe which way the aircraft moves — that isolated one question from the entire feedback loop.

Bug 2 — Raw inputs exploded the recurrent state

With PID fixed, the LSTM retrained beautifully — imitation loss of 0.00142, a thirty-fold improvement. But RTRL was still emitting a constant, even *before* any online learning had run. The warm-started network was already broken.

The cause was architectural. The RTU's recurrence is deliberately linear and unbounded — that is precisely what keeps RTRL's mathematics exact. But an LSTM contains squashing functions that quietly bound whatever passes through them. The LSTM had been trained on raw observations including altitude around 5,000 feet, and its internal gates absorbed that magnitude harmlessly.

The RTU had no such protection. Feeding it a value of 5,000 through the inherited weights drove its hidden state to a magnitude of about **28,000 within thirty steps**. Everything downstream saturated.

The fix: have the RTU normalise its own inputs internally, so it is protected regardless of what the caller provides. Hidden state magnitude dropped from 28,000 to about 9.

THE TEACHING POINT

When you transfer weights between architectures, you transfer them into a different set of assumptions. The LSTM's tolerance for large inputs was a property of its *nonlinearities*, not its weights — and that tolerance did not come along with the transfer.

Bug 3 — Non-finite rewards permanently poisoned the network

During training, the aircraft occasionally departed controlled flight badly enough that the simulator produced a non-finite reward. That value entered the learning step and corrupted the network's parameters irrecoverably. From that point the network was mathematically dead for the rest of the run.

Two fixes. Immediately: refuse to learn from a non-finite value — skip that single step. Properly (deferred as out of

scope): the task should terminate an episode when the aircraft exceeds safe attitude bounds, rather than continuing to simulate a departed aircraft.

THE TEACHING POINT

Any value flowing from a simulator into a learning update needs a finiteness check. One bad number, applied once, can destroy hours of training with no error message.

Bug 4 — Systematic weight drift

After the first three fixes, RTRL trained cleanly for fifty episodes with no crashes. And still saturated. Tracing the parameters revealed steady, monotonic growth in the output weights across training. Several standard remedies were tried in sequence — normalising the reward signal, lowering the learning rate, tightening gradient clipping, adding weight decay, imposing a hard cap on weight magnitude. Each slowed the drift. None stopped it. The weights simply took longer to reach saturation. That pattern — where every magnitude-limiting technique delays but does not prevent the failure — indicates the problem is not magnitude at all. It is *direction*.

7.8 Stage 8 — The real fix

The actual defect was in how learning credit was distributed among the three outputs. The original design computed a single number measuring how well the aircraft was doing, and handed that same number to all three output channels as their learning signal. Aileron, elevator, and rudder each received an identical instruction, every timestep, roughly fifteen thousand times. Consider what that means. Suppose the aircraft is doing badly because pitch is off. All three channels are told "you are doing badly." The elevator — genuinely responsible — adjusts appropriately. But the aileron and rudder receive the same signal despite having nothing to do with the problem, and adjust in whatever direction their internal structure happens to push them. That direction is the same every time. Fifteen thousand consistent nudges in one direction will saturate any parameter, regardless of how small each nudge is.

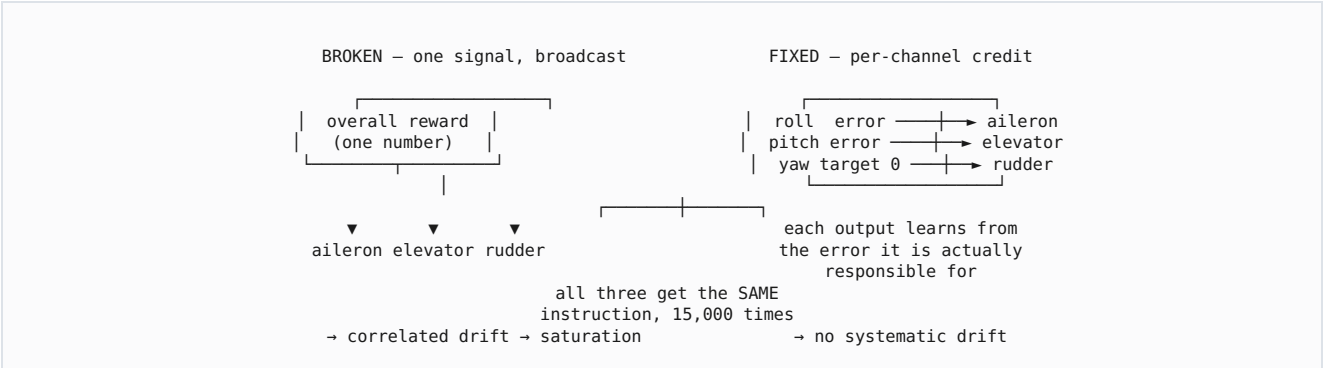


Figure 7.1 — The bug that mattered, and its correction.

The fix gave each output its own target. The aileron is responsible for roll, so it learns from roll error. The elevator is responsible for pitch, so it learns from pitch error. The rudder targets zero yaw. Each output now receives a signal about the thing it actually controls.

The result was immediate and unambiguous:

Measurement	Before	After
Sensitivity magnitude over training	Grew steadily, 166 → 577	Flat at ~110 throughout
Output weight magnitude	Grew to 9.6 despite caps	Stable below 1.0, no caps needed
Control jerk (fault scenario)	0.0000 — constant output	0.1159 — actively controlling
Magnitude-limiting hacks required	Weight decay + hard norm cap	None — all removed

The clearest evidence that this was the true root cause is the last row. Once credit assignment was correct, every band-aid added to suppress the symptom became unnecessary and was deleted.

THE CENTRAL ENGINEERING LESSON OF THIS PROJECT

When a symptom resists every technique aimed at limiting its magnitude, the problem is not magnitude. Stop adding constraints and find the systematic bias. Four separate remedies were applied to a symptom before the cause was correctly identified — each one delaying the failure rather than preventing it, and each one making the true diagnosis slightly harder to see.

7.9 Stage 9 — Full sweep and results

With the controllers working, the full experiment matrix ran: three controllers across four scenarios with three random seeds each, plus the three-seed ablation. Thirty-nine episodes, all completed cleanly.

One more bug surfaced here, worth mentioning because it is so common. Random seeds were being passed through configuration but never actually reaching the simulator — and even once that was fixed, the task was sampling its targets from a different random source that the seed did not control. Two separate breaks in the same chain. Both fixed, so that seeds now produce genuinely distinct and reproducible episodes.

A final health check verified everything: tests green, formatting clean, no hardcoded paths, all figures generated, and — importantly — three design documents corrected where they had drifted behind the implementation.

Part 8 — What the Experiments Showed

8.1 The headline result

THE ABLATION

On the fault scenario, the RTRL controller with online updates **enabled** achieved an attitude RMSE of **1.53 radians**. The identical controller, with the identical starting weights, on the identical scenario, with online updates **disabled**, achieved **2.05 radians**.

That is roughly a **25% improvement**, attributable to online adaptation and nothing else.

This is the number the project was built to produce. Because the two conditions differ in exactly one respect, there is no alternative explanation available. Online weight adaptation during flight measurably improved fault performance.

8.2 Reading the numbers honestly

The rest of the results require more careful reading, and the honest reading is less flattering.

PID was the strongest controller overall. Once its sign error was fixed and its gains retuned, it outperformed both neural controllers on tracking accuracy across every scenario. This should be stated plainly rather than buried.

BPTT-LSTM outperformed RTRL-RTU on absolute tracking. Also true, also worth stating. But the comparison is not as meaningful as it appears, for a specific reason: the LSTM received two hundred episodes of supervised imitation from a well-tuned teacher, while RTRL received fifty episodes with no teacher signal at deployment and a learning objective that is a surrogate rather than the true reward. The two received very different amounts of training investment. The comparison measures effort at least as much as it measures method.

RTRL-RTU is now genuinely controlling the aircraft. Its control jerk of 0.116 on the fault scenario — against exactly zero before the credit-assignment fix — confirms it responds to input rather than emitting a constant. Its RMSE of 1.67 radians is worse than the other two but far better than the 2.15 it produced when saturated.

WHY THE HONEST FRAMING IS THE STRONGER FRAMING

A report claiming RTRL beat everything would invite immediate scepticism, and the scepticism would be justified. A report that says "here is the one comparison I designed to be clean, here is what it shows, and here is exactly why the other comparisons are confounded" is far more credible — and demonstrates better experimental thinking than a better-looking result obtained carelessly.

8.3 The five figures

Figure	What it shows
nominal_tracking	Tracking error per controller in calm air, with error bars across three seeds. Establishes the baseline.
fault_recovery	Pitch and roll error over time for all three controllers during a fault episode, with fault onset marked. The most informative single figure — it shows the shape of each controller's response, not just a summary number.
wind_robustness	Tracking error under turbulence never seen in training.
combined_stress	Fault and turbulence together.
ablation	Online-enabled versus online-disabled on the fault scenario. <i>The paper's central claim, in one image.</i>

Part 9 — Lessons, Limitations, and Next Steps

9.1 What this project proves

- **Modern RTRL is practical for flight control.** Using an element-wise recurrent architecture, exact RTRL runs comfortably at simulation rate on ordinary hardware. The historical objection — that RTRL is too expensive to use — no longer applies with the right architecture.
- **Online adaptation measurably helps under fault conditions.** The ablation is clean and the effect is real: a 25% reduction in tracking error from adaptation alone.
- **The implementation is verifiably correct.** The sensitivity computation agrees with two independent references to five decimal places. Whatever else is true, the mathematics is right.

9.2 What it does not prove

- **That RTRL is the best flight controller.** It is not, in this study. A well-tuned PID beat it on every scenario.
- **That the result is statistically robust.** Three seeds per condition is thin. The direction of the ablation effect is consistent, but the confidence interval is wide. This is a proof of concept, not a definitive measurement.
- **That it would work on real hardware.** Everything here is simulation. The gap between JSBSim and a real airframe is substantial and well documented in the sim-to-real literature.
- **That the online objective is well designed.** The per-channel tracking loss is a surrogate — a reasonable one, but not derived from the true reward. A better objective is the most promising direction for improvement.

9.3 Engineering lessons worth keeping

Lesson	Where it came from
Verify the critical component first, in isolation	The sensitivity test was written on day five, before RTRL met the simulator. It meant the mathematics was never a suspect during later debugging.
Probe your dependencies before designing against them	Thirty minutes of asking the simulator what it actually provides saved days of building against an assumption that turned out to be false.
Passing tests are not working systems	Forty-four green tests coexisted with three controllers that could not fly. Tests verify components; only experiments verify systems.
When more authority makes things worse, check the sign	The PID sign inversion. No tuning could have fixed it, and tuning was where the effort initially went.
When magnitude limits only delay a failure, look for directional bias	Four separate remedies were applied to the saturation symptom before the credit-assignment cause was found. Each delayed the failure and obscured the diagnosis.
Zero can be a red flag	Control jerk of exactly 0.0000 looked like perfect smoothness and was in fact total collapse.
Write down what you will not build	The prohibition list in <code>CLAUDE.md</code> held the scope through fourteen days of tempting extensions.
Fix documents when they drift	Three design documents had fallen behind the code and were corrected at the final check. A document that lies is worse than no document.

9.4 If you had four more weeks

In rough order of value per unit effort:

1. **Add safe-flight termination to the task.** End an episode when attitude exceeds sensible bounds. This eliminates the non-finite reward pathology at its source rather than guarding against it, and would make training substantially more stable.
2. **Increase to ten seeds per condition.** The cheapest possible improvement to statistical credibility. Pure compute time.
3. **Improve the online objective.** The per-channel tracking loss is a surrogate. A properly derived policy-gradient objective, or a value-based critic, would likely close much of the gap to the LSTM.
4. **Match the training budgets.** Give RTRL the same two hundred episodes the LSTM received, so the comparison measures method rather than effort.

5. **Sweep fault severity.** Currently one severity is tested. A curve of recovery quality against severity would show *where* adaptation helps most — likely a more interesting result than a single point.
6. **Add a second airframe.** Demonstrating the approach generalises beyond the Cessna 172 would considerably strengthen the claim.

Part 10 — Glossary

Term	Meaning
Ablation	An experiment where one component is removed while everything else is held fixed, to isolate that component's contribution.
Aileron	Wing control surface governing roll.
Attitude	The aircraft's orientation in space — pitch, roll, and yaw angles.
BPTT	Backpropagation Through Time. The standard way to train recurrent networks: unroll the history, then compute gradients backward from the end.
Control jerk	How rapidly the control commands change from step to step. A smoothness measure — but a value of zero indicates a stuck controller, not a smooth one.
Dryden turbulence	A standard statistical model of atmospheric turbulence used in flight simulation.
Element-wise recurrence	A recurrent architecture where each hidden unit evolves independently of the others. Less expressive than dense coupling, but it is what makes exact RTRL affordable.
Elevator	Tail control surface governing pitch.
Hidden state	A recurrent network's internal memory, carried from one timestep to the next.
Imitation learning	Training a network by showing it an expert's inputs and outputs and teaching it to reproduce them.
JSBSim	An open-source, high-fidelity flight dynamics model. The physics engine used throughout this project.
PID	Proportional-Integral-Derivative. The classical control algorithm: respond to current error, accumulated error, and rate of change of error.
RMSE	Root Mean Square Error. The primary accuracy measure here — average tracking error across an episode.
RTRL	Real-Time Recurrent Learning. Trains recurrent networks by carrying gradient information forward, enabling weight updates at any timestep — including mid-flight.
RTU	Recurrent Trace Unit. The element-wise recurrent architecture used here, from Elelimy et al. (2024).
Rudder	Tail control surface governing yaw.
Sensitivity matrix	RTRL's central quantity. Tracks how the current hidden state would change in response to a nudge in each weight, carried forward each timestep.
Settling time	How long a controller takes to bring error below a threshold and keep it there.
Warm start	Initialising a network from another trained network's weights rather than from random values.
Wrapper	A layer placed around a simulation environment that intercepts and modifies information passing through it — used here for faults, wind, normalisation, and logging.